

UNIT TESTING USANDO UN LLM

Michele Marchesi

marchesi@unica.it

**Corso di Ingegneria del Software
A.A. 2025/26**

Presentazione inizialmente generata da ChatGPT 5



Perché usare un LLM per i test unitari

- Riduce il tempo per creare una prima bozza di test.
- Suggerisce casi positivi, negativi e casi limite che si possono trascurare.
- Aiuta a costruire rapidamente fixture, mock object e parametrizzazioni.
- È utile soprattutto su codice legacy o librerie di funzioni o classi prive di test.
- Funziona meglio come “pair programmer” che come generatore da accettare alla cieca.



Flusso operativo consigliato

- 1. Fornisci contesto**
incolla funzione/classe, dipendenze, obiettivo e framework di test desiderato
- 2. Chiedi i casi di test**
happy path, errori, edge case, parametri limite, eccezioni
- 3. Genera i test**
pytest o unittest, con mock/fixture dove servono
- 4. Esegui e correggi**
run locale, fallimenti, import mancanti, assunzioni errate
- 5. Raffina la suite**
naming, coverage, robustezza, leggibilità e manutenzione



Che cosa dare in input al modello

Minimo indispensabile

- codice da testare
- comportamento atteso
- framework preferito (pytest/unittest)
- versione Python
- vincoli: no I/O reale, usa mock object, gestisci eccezioni

Informazioni utili in più

- esempi di input/output
- tipi previsti
- dipendenze esterne da simulare
- stile del progetto
- file già presenti (conftest.py, fixtures, helpers)

Prompt base:

Scrivi unit test pytest per questa funzione Python.
Copri: caso nominale, input non validi, edge case.
Non usare rete o DB reali. Usa unittest.mock se necessario.
Restituisci solo il file test_*.py.



Esempio: codice Python da testare

```
1 def normalize_price(raw: str) -> float:
2     if raw is None:
3         raise ValueError("missing price")
4     text = raw.strip().replace("€", "").replace(",", ".")
5     if not text:
6         raise ValueError("empty price")
7     value = float(text)
8     if value < 0:
9         raise ValueError("negative price")
10    return round(value, 2)
```

Aspetti da testare:

- input valido con simbolo €
- virgola decimale
- spazi iniziali/finali
- stringa vuota
- None
- valore negativo

Richiesta al LLM

“Genera test pytest compatti, con nomi chiari, coprendo eccezioni e casi limite.”

Output atteso: bozza di test generata

```
1 import pytest
2 from app.pricing import normalize_price
3
4 def test_normalize_price_ok():
5     assert normalize_price(" €12,50 ") == 12.50
6
7 @pytest.mark.parametrize("raw", ["", " ", None])
8 def test_normalize_price_invalid(raw):
9     with pytest.raises(ValueError):
10         normalize_price(raw)
11
12 def test_normalize_price_negative():
13     with pytest.raises(ValueError):
14         normalize_price("-1")
```

- Buona base iniziale, ma va controllata.
- Potrebbe mancare un caso su input non numerico.
- Percorsi di import e messaggi d'errore vanno allineati al progetto reale.

Step successivo:

Eseguire i test, correggere assunzioni errate e rifinire la suite.



Come chiedere test migliori

- Specifica il framework: pytest è spesso il più comodo per generazione e lettura.
- Chiedi esplicitamente parametrizzazione, fixture e mocking dove utili.
- Imponi vincoli: nessun accesso a rete, file system o database reale.
- Chiedi di coprire errori, edge case e regressioni note.
- Fai generare solo il codice finale oppure prima l'elenco dei casi di test.



1 Prima elenca i casi di test in 6 bullet.
2 Poi genera i test pytest.
3 Usa parametrize quando appropriato.
4 Se una dipendenza esterna è coinvolta, usa unittest.mock.

Regola pratica:

Meglio prompt specifici e iterativi che un'unica richiesta troppo generica.

I principali rischi

Assunzioni sbagliate

import errati,
comportamento inventato,
eccezioni non reali

Coverage solo apparente

test che passano ma non
verificano davvero il codice

Fragilità

asserzioni troppo
specifiche o dipendenti
dall'implementazione

- Un LLM può scrivere test plausibili ma inutili.
- Serve sempre esecuzione reale della suite e review umana.
- Conviene partire da unit test semplici prima di testare integrazioni o UI.
- Il team deve considerare il modello come assistente, non come fonte di verità.



Buone pratiche operative

- Validare sempre il test eseguendolo localmente o in CI.
- Far generare prima la lista dei casi e solo dopo il codice.
- Mantenere test piccoli, leggibili e indipendenti.
- Usare fixture condivise e mock object espliciti per ridurre le duplicazioni.
- Misurare il valore: più velocità di scrittura, meno omissioni, qualità invariata o migliore.

Pipeline tipica

Sorgente → prompt →
bozza test → run pytest →
fix → review → commit

Output ideale

Test corretti, leggibili e facili
da mantenere.



Conclusioni

1

Un LLM accelera la creazione della prima suite di test.

2

Funziona bene se il prompt descrive codice, contratto e vincoli.

3

La verifica reale con pytest e review umana resta indispensabile.

- Approccio consigliato: iniziare con piccoli esempi, usare iterazioni rapide e correzioni guidate dai fallimenti.
- Partire da codice puro o facilmente rappresentabile con mock object massimizza il rapporto valore/sforzo.
- Obiettivo pratico: generare test utili più velocemente, non delegare la qualità al modello.

